Name: Aaditya Surve

Roll No.: 2401119

Class: T2 Batch: T22

ASD&DO Experiment No. 09

Lab outcomes :

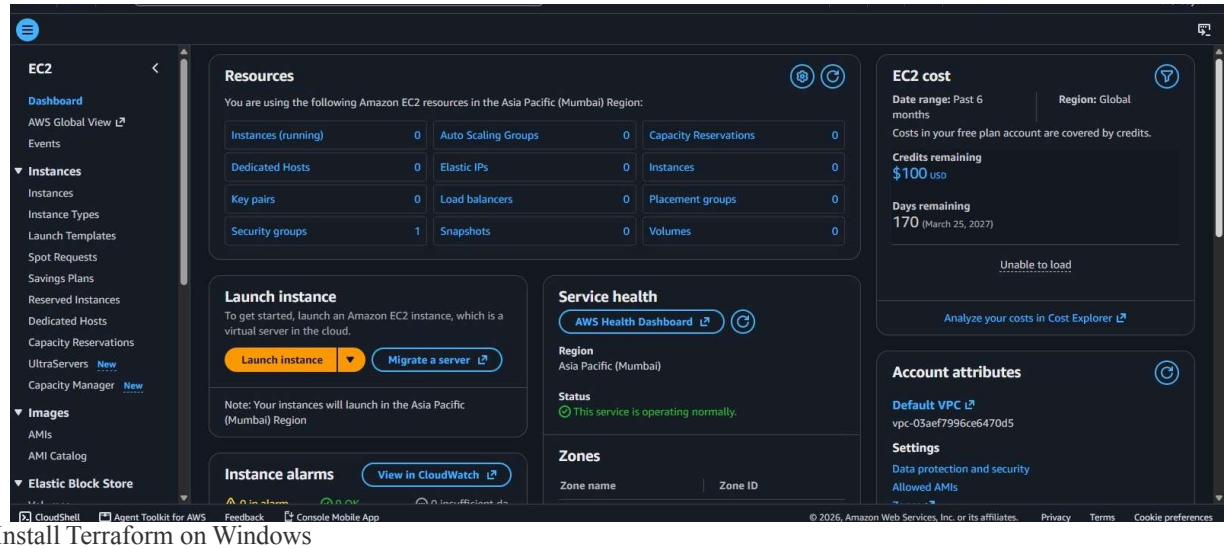
LO4: Containerize, orchestrate, and deploy applications using Docker, Docker Compose, Kubernetes, and cloud infrastructure tools.

Theory:

Step 1: Create or verify your AWS account

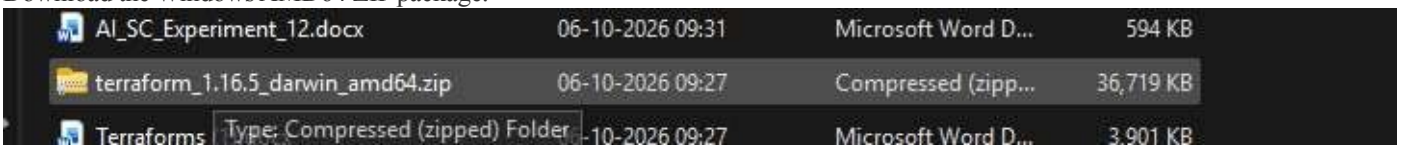
1. Open the AWS Management Console .
2. Sign in to your AWS account.
3. Select the Mumbai region (ap-south-1).
4. Open the EC2 Dashboard and check that your account is ready to launch instances.

The account must have permissions to create EC2 instances and use the required networking resources.

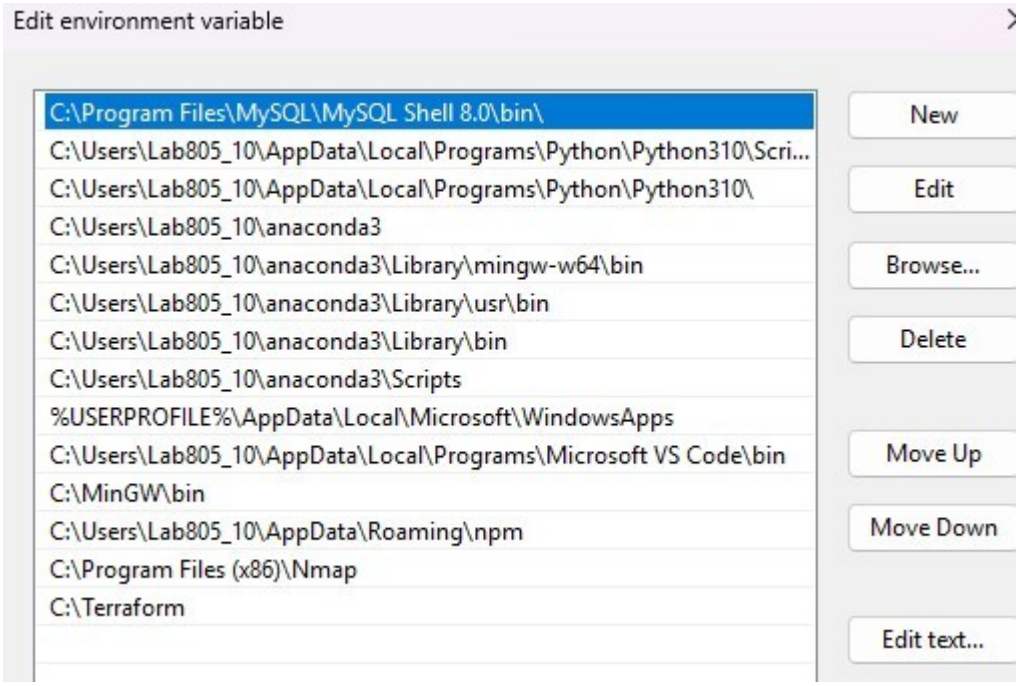


Step 2: Install Terraform on Windows

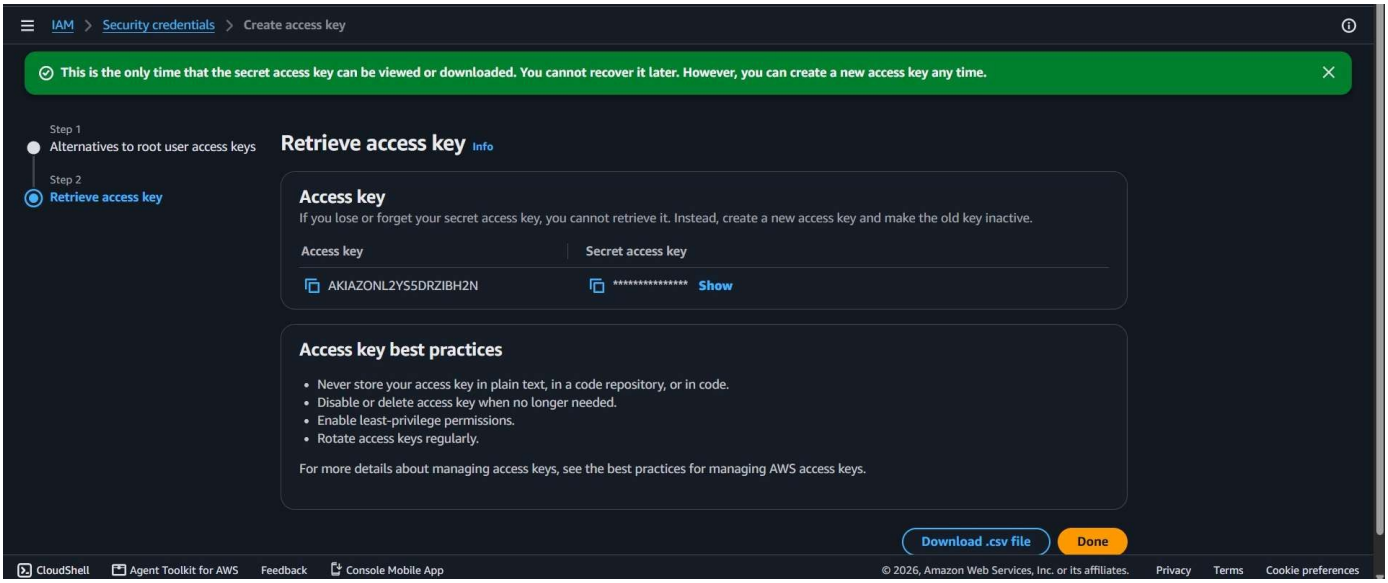
1. Visit the official Terraform installation page .
2. Download the Windows AMD64 ZIP package.



3. Extract terraform.exe into a folder, for example: C:\Terraform
4. Add C:\Terraform to the Windows PATH environment variable.



5. Open a new Command Prompt or PowerShell window.




```
C:\Users\Lab805_10>terraform -version
Terraform v1.16.4
on windows_amd64

Your version of Terraform is out of date! The latest version
is 1.16.5. You can update by downloading from https://developer.hashicorp.com/terraform/install

C:\Users\Lab805_10>
```

```
PS D:\terraform_1.16.5_windows_386> dir


    Directory: D:\terraform_1.16.5_windows_386


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----         10/8/2026  11:10 PM           4950 LICENSE.txt
-a----         10/8/2026  11:10 PM      116640904 terraform.exe


PS D:\terraform_1.16.5_windows_386> notepad main.tf
```

main.tf lec1 web dev.txt

File Edit View

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}

provider "aws" {
  region = "us-east-1"
}

resource "aws_instance" "demo" {
  ami           = "ami-0c02fb55956c7d316"
  instance_type = "t2.micro"

  tags = {
    Name = "Terraform-Demo"
  }
}
```

```
PS D:\terraform_1.16.5_windows_386> terraform init
Initializing the backend...

Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 6.0"...
- Installing hashicorp/aws v6.68.0...
- Installed hashicorp/aws v6.68.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS D:\terraform_1.16.5_windows_386> terraform apply
```

```
PS D:\terraform_1.16.5_windows_386> terraform validate
Success! The configuration is valid.

PS D:\terraform_1.16.5_windows_386> terraform plan

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the
following symbols:
  + create

Terraform will perform the following actions:

  # aws_instance.demo will be created
  + resource "aws_instance" "demo" {
      + ami                                  = "ami-0c02fb55956c7d316"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = (known after apply)
      + availability_zone                    = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
      + ebs_optimized                        = (known after apply)
      + enable_primary_ipv6                  = (known after apply)
      + force_destroy                        = false
      + get_password_data                    = false
      + host_id                              = (known after apply)
      + host_resource_group_arn              = (known after apply)
      + iam_instance_profile                 = (known after apply)
      + id                                   = (known after apply)
      + instance_initiated_shutdown_behavior = (known after apply)
      + instance_lifecycle                   = (known after apply)
      + instance_state                       = (known after apply)
      + instance_type                        = "t2.micro"
      + ipv6_address_count                   = (known after apply)
      + ipv6_addresses                       = (known after apply)
      + key_name                             = (known after apply)
      + monitoring                           = (known after apply)
      + outpost_arn                          = (known after apply)
      + password_data                        = (known after apply)
      + placement_group                      = (known after apply)
      + placement_group_id                   = (known after apply)
      + placement_partition_number           = (known after apply)
```

Windows PowerShell

```
      + capacity_reservation_specification (known after apply)

      + cpu_options (known after apply)

      + ebs_block_device (known after apply)

      + enclave_options (known after apply)

      + ephemeral_block_device (known after apply)

      + instance_market_options (known after apply)

      + maintenance_options (known after apply)

      + metadata_options (known after apply)

      + network_interface (known after apply)

      + primary_network_interface (known after apply)

      + private_dns_name_options (known after apply)

      + root_block_device (known after apply)

      + secondary_network_interface (known after apply)
    }

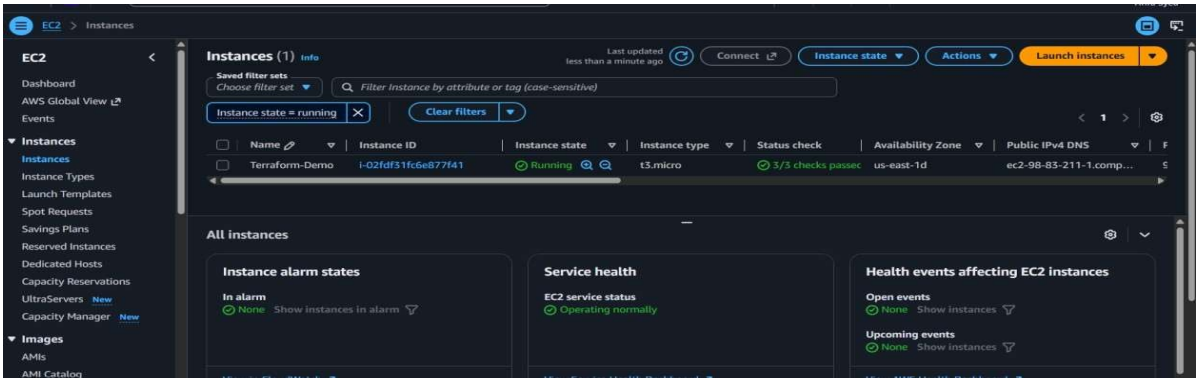
Plan: 1 to add, 0 to change, 0 to destroy.

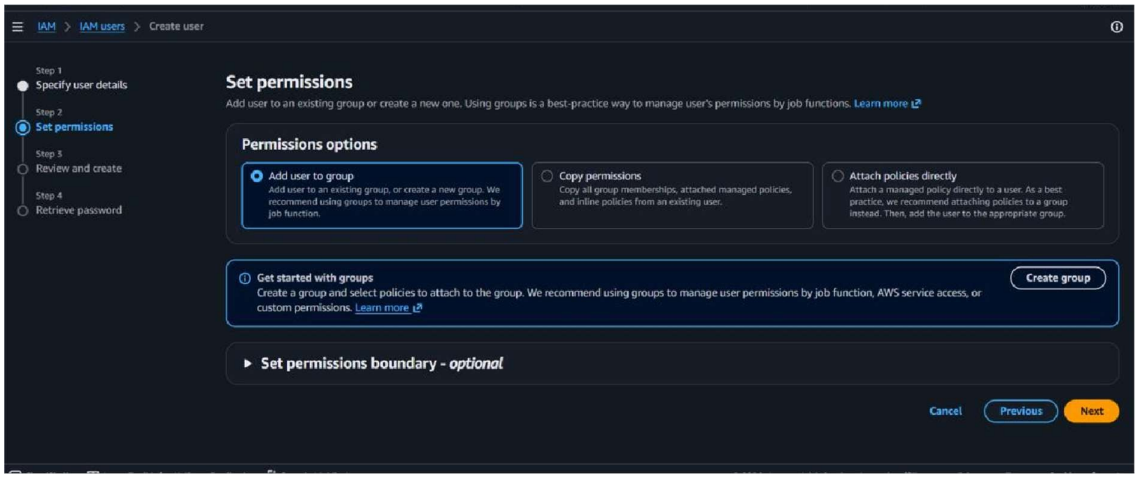
Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_instance.demo: Creating...
aws_instance.demo: Still creating... [00m10s elapsed]
aws_instance.demo: Creation complete after 17s [id=i-02fdf31fc6e877f41]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
PS D:\terraform_1.16.5_windows_386>
```






```
PS D:\terraform_1.16.5_windows_386> terraform apply
aws_instance.demo: Refreshing state... [id=i-0643ee2c65bebe2e5]

Changes to Outputs:
  + instance_id   = "i-0643ee2c65bebe2e5"
  + instance_type = "t3.micro"
  + public_ip     = "54.80.247.205"

You can apply this plan to save these new output values to the Terraform state, without changing any real infrastructure.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes


Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

instance_id = "i-0643ee2c65bebe2e5"
instance_type = "t3.micro"
public_ip = "54.80.247.205"
```

.tf files in the same folder are treated as **one Terraform configuration**. The filename does not determine what gets executed.

Output:

12. Learning Outcomes Achieved :

Students gained practical knowledge of using Terraform to define, provision, and manage AWS cloud infrastructure through Infrastructure as Code (IaC).

13. Conclusion:

1. **Applications of the studied technique in industry**
   Terraform enables automated and consistent provisioning of cloud infrastructure, supporting efficient deployment and management of AWS resources.

2. **Engineering Relevance**
   The experiment demonstrates Infrastructure as Code practices that improve scalability, reliability, repeatability, and efficiency in cloud-based engineering.

3. **Skills Developed**
   Students developed skills in writing Terraform configurations, executing Terraform commands, and provisioning AWS resources using Terraform.